

```

corpus = [
    "the girl played with the toy",
    "the girl bought a toy",
    "the girl bought a chocolate",
    "the boy ate the chocolate"
]

# Tokenize
sentences = [sentence.lower().split() for sentence in corpus]

bigram_counts = {}
unigram_counts = {}

for sentence in sentences:
    for i in range(len(sentence)):
        w1 = sentence[i]
        unigram_counts[w1] = unigram_counts.get(w1, 0) + 1

        if i < len(sentence) - 1:
            w2 = sentence[i+1]
            if w1 not in bigram_counts:
                bigram_counts[w1] = {}
            bigram_counts[w1][w2] = bigram_counts[w1].get(w2, 0) + 1

def predict_next(word):
    if word not in bigram_counts:
        return "No prediction (word not in vocabulary)"
    next_words = bigram_counts[word]
    prediction = max(next_words, key=next_words.get)
    return prediction

# Test predictions
print(predict_next("the"))           # likely "girl"
print(predict_next("girl"))          # likely "bought"
print(predict_next("chocolate"))    # No prediction (end of sentence)

girl
bought
No prediction (word not in vocabulary)

import nltk
from nltk.util import ngrams
from collections import Counter

corpus = [
    "the girl played with the toy",
    "the girl bought a toy",
    "the girl bought a chocolate",
    "the boy ate the chocolate"
]

```

```

# Tokenize
tokens = []
for sentence in corpus:
    tokens.extend(sentence.lower().split())

# Build n-grams
bigrams = list(ngrams(tokens, 2))
unigrams = list(ngrams(tokens, 1))

# Count frequencies
bigram_counts = Counter(bigrams)
unigram_counts = Counter(unigrams)

def predict_next(word):
    # Find all bigrams starting with the given word
    candidates = {w2: count for (w1, w2), count in
bigram_counts.items() if w1 == word}

    if not candidates:
        return "No prediction (word not in vocabulary)"

    # Pick the most frequent next word
    prediction = max(candidates, key=candidates.get)
    return prediction

# Test predictions
print(predict_next("the"))      # likely "girl"
print(predict_next("girl"))    # likely "bought"
print(predict_next("boy"))     # likely "ate"
print(predict_next("chocolate"))# No prediction (end of sentence)

girl
bought
ate
the

from collections import Counter
from nltk.util import ngrams
import random

corpus = [
    "the girl played with the toy",
    "the girl bought a toy",
    "the girl bought a chocolate",
    "the boy ate the chocolate"
]

# Tokenize and lowercase
tokens = []
for sentence in corpus:

```

```

tokens.extend(sentence.lower().split())

# Build trigrams and bigrams
trigrams = list(ngrams(tokens, 3))
bigrams = [(w1, w2) for (w1, w2, w3) in trigrams]

trigram_freq = Counter(trigrams)
bigram_freq = Counter(bigrams)

# Predict next word (deterministic)
def predict_next(w1, w2):
    candidates = {w3: freq for (a, b, w3), freq in
trigram_freq.items() if a == w1 and b == w2}
    if not candidates:
        return "No prediction"
    return max(candidates, key=candidates.get)

# Optional: Predict next word (probabilistic)
def predict_next_random(w1, w2):
    candidates = [(w3, freq) for (a, b, w3), freq in
trigram_freq.items() if a == w1 and b == w2]
    if not candidates:
        return "No prediction"
    words, weights = zip(*candidates)
    return random.choices(words, weights=weights, k=1)[0]

# Test predictions
print("Next word after ('the','girl'):", predict_next("the", "girl"))
print("Next word after ('girl','bought'):", predict_next("girl",
"bought"))
print("Next word after ('the','boy'):", predict_next("the", "boy"))

Next word after ('the','girl'): bought
Next word after ('girl','bought'): a
Next word after ('the','boy'): ate

```